



Java — M.Tech Exam-Ready Notes (Units I–V)

Core Java → Servlets/JSP → RMI/EJB — with Table of Contents & Clickable GFG Links

How to use this file: Every topic has (1) a one-line definition to open your answer with, (2) the explanation in plain English (what to *write*), and (3) code/diagrams as supporting evidence. Each section also has a **GFG** link for extra reading/practice problems.

Note on the links: These are formatted as GeeksforGeeks *search-result* links ([geeksforgeeks.org/?s=...](https://www.geeksforgeeks.org/?s=...)) rather than single guessed article URLs — clicking one takes you straight to GFG's top result for that exact topic. This is deliberate: it guarantees a working, relevant page every time instead of risking a dead link to a guessed URL. A couple of the highest-traffic topics below are marked **[Verified]** where I confirmed the exact canonical article.

In an exam, always start with the definition, then explain in 3–5 lines, then give a small code snippet or diagram.



Table of Contents

- [Syllabus Map](#)
- **UNIT I — Review of Java Concepts**
 - [1.1 Features of Java](#)
 - [1.2 Object-Oriented Programming Overview](#)
 - [1.3 Introduction to Java Technologies \(JDK/JRE/JVM\)](#)
 - [1.4 Writing a Simple Java Program](#)
 - [1.5 Data Types](#)
 - [1.6 Variables & Memory Concepts](#)
 - [1.7 Control Statements](#)
 - [1.8 Looping Constructs](#)
 - [1.9 Method Call Stack & Activation Records](#)
 - [1.10 Argument Promotion and Casting](#)
 - [1.11 Scope of Declaration and Method Overloading](#)
 - [1.12 String Handling](#)
 - [1.13 Arrays](#)
- **UNIT II — Inheritance, Packages, Exceptions, Threads, Applets, JDBC**
 - [2.1 Inheritance: Extending Classes](#)
 - [2.2 Packages and Interfaces](#)
 - [2.3 Exception Handling](#)
 - [2.4 Multithreading](#)
 - [2.5 Applets](#)
 - [2.6 Database Connectivity: JDBC](#)
- **UNIT III — HTTP, Web Servers & Servlets**
 - [3.1 Introduction to HTTP, Web Servers, and Application Servers](#)
 - [3.2 Java Servlets](#)
 - [3.3 Servlet Packages, Classes, Interfaces & Methods](#)

- 3.4 Handling Forms with a Servlet
- 3.5 Session Handling — Various Methods
- 3.6 Elements of Deployment Descriptors
- **UNIT IV — JSP & Database-Backed Web Apps**
 - 4.1 JSP Basics
 - 4.2 JSP/Servlet Database Connectivity — Oracle, MS-SQL, MySQL
 - 4.3 Separating Business Logic and Presentation Logic (JavaBeans)
 - 4.4 Session Handling in JSP
 - 4.5 Types of Errors and Exception Handling (JSP)
- **UNIT V — MVC, RMI & EJB**
 - 5.1 MVC Architecture
 - 5.2 Introduction to Remote Method Invocation (RMI)
 - 5.3 Introduction to Enterprise JavaBeans (EJB)
-  Recommended Resources (Books + Official Docs)
-  Quick Exam-Day Revision Checklist

Syllabus Map

Unit	Hours	Focus
I	8	Core Java refresher — syntax, control flow, strings, arrays
II	8	Inheritance, Packages, Exceptions, Multithreading, Applets, JDBC
III	8	HTTP/Web & App Servers, Servlets
IV	8	JSP, DB connectivity, JavaBeans
V	8	MVC, RMI, EJB

[↑ TOC](#)

UNIT-I: Review of Java Concepts

1.1 Features of Java

Definition to write first: *Java is a high-level, object-oriented, platform-independent programming language designed to be simple, secure, and robust.*

Feature	Meaning (explain like this in exam)
Simple	Removes complex features of C++ like pointers, operator overloading, and multiple inheritance of classes, making it easier to learn and maintain.
Object-Oriented	Everything (except the 8 primitive types) is treated as an object — this enables reusability via inheritance, encapsulation, and polymorphism.

Feature	Meaning (explain like this in exam)
Platform Independent	Follows the principle " Write Once, Run Anywhere (WORA) " — source code compiles to an intermediate form called <i>bytecode</i> , which any machine with a JVM can execute, regardless of the underlying OS/hardware.
Secure	No explicit pointers (prevents illegal memory access), bytecode is verified before execution, and code runs inside a sandbox controlled by the Security Manager/ClassLoader.
Robust	Strong compile-time type checking + automatic garbage collection (no manual memory management like <code>malloc/free</code>) + mandatory exception handling reduce runtime crashes.
Multithreaded	Built-in language-level support (the <code>Thread</code> class, <code>Runnable</code> interface) lets a program perform multiple tasks concurrently.
Architecture-Neutral	The bytecode format itself does not depend on any specific CPU architecture.
Portable	Primitive data type sizes are fixed by the language specification (e.g., <code>int</code> is always 32-bit everywhere), unlike C/C++ where sizes vary by compiler/OS.
High Performance	The JIT (Just-In-Time) compiler converts frequently executed bytecode into native machine code at runtime, giving near-native speed.
Distributed	Comes with built-in networking libraries (<code>java.net</code>) and RMI, making it easy to build applications that work across a network.
Dynamic	Classes are loaded into memory only when needed (dynamic class loading), and Java supports <i>reflection</i> — inspecting/modifying classes at runtime.

Exam tip: If asked "Explain features of Java" for 5+ marks, pick the top 6 (Simple, OOP, Platform Independent, Robust, Secure, Multithreaded) and write 2 lines each — that alone covers most weightage.

 **GFG: Features of Java**

[↑ TOC](#)

1.2 Object-Oriented Programming Overview

The four pillars of OOP (always list all four even if asked about only one, briefly):

1. **Encapsulation** — binding data (fields) and methods together into a single unit (class), and restricting direct access to internal data using access modifiers (`private`), exposing controlled access via getters/setters.
2. **Inheritance** — a mechanism where a new class (subclass) acquires the properties and behavior of an existing class (superclass), promoting code reuse.
3. **Polymorphism** — the ability of an object/method to take many forms; in Java this appears as **method overloading** (compile-time/static polymorphism) and **method overriding** (runtime/dynamic polymorphism).

4. **Abstraction** — hiding implementation details and showing only the essential features, achieved in Java through **abstract** classes and **interfaces**.

Java-specific OOP notes (good for "compare Java and C++ OOP" type questions):

- Java enforces OOP more strictly than C++ — there are no free/global functions; every method must belong to a class.
- Java supports **single inheritance for classes** (a class can extend only one class) but **multiple inheritance is allowed through interfaces**.
- All non-static, non-final, non-private methods are *virtual* by default, meaning method calls are resolved at runtime based on the actual object type (dynamic method dispatch).

 **GFG: OOPs Concepts in Java**

[↑ TOC](#)

1.3 Introduction to Java Technologies

Explain the relationship like this:

```
JDK (Java Development Kit) = JRE + development tools (javac, javadoc, jar,
jdb, jshell)
                                → needed to DEVELOP and RUN Java programs
JRE (Java Runtime Env.)    = JVM + core class libraries
                                → needed only to RUN Java programs (no
compiler)
JVM (Java Virtual Machine) = the engine that loads, verifies, and executes
bytecode
                                → platform-specific implementation
(different JVM per OS)
```

Relationship to remember: $JDK \supset JRE \supset JVM$ (JDK is the superset).

Java Editions (good for a short-answer question "types of Java platforms"):

```
Java SE (Standard Edition) → core language + desktop/console apps
[Unit I & II]
Java EE / Jakarta EE      → enterprise APIs: Servlets, JSP, EJB
[Units III-V]
Java ME (Micro Edition)   → embedded and mobile devices (legacy,
largely replaced by Android)
```

Compilation & Execution Flow (draw this diagram in exam):

```
MyProgram.java --[javac compiler]--> MyProgram.class (bytecode) --[java command / JVM]--> Output
```

Explanation: `javac` translates human-readable source code into **bytecode** (a `.class` file, not machine code). The JVM then interprets/JIT-compiles this bytecode into native instructions for the host machine — this two-step process is *why* Java is platform-independent.

 **GFG:** [JDK vs JRE vs JVM](#)

[↑ TOC](#)

1.4 Writing a Simple Java Program

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

```
javac HelloWorld.java      # compiles to HelloWorld.class  
java HelloWorld           # runs it (JVM loads HelloWorld.class)
```

Key rules to state in exam (common 2-mark question: "rules for main method"):

- The `public class` name **must exactly match** the filename (`HelloWorld.java` ↔ `class HelloWorld`).
- `main()` is the JVM's entry point and its signature must be exactly `public static void main(String[] args)`:
 - `public` — so JVM (outside the class) can call it.
 - `static` — so JVM can call it without creating an object first.
 - `void` — it does not return anything to the JVM.
 - `String[] args` — accepts command-line arguments as an array of strings.
- `System.out` is a `PrintStream` object representing the standard output stream (the console).

 **GFG:** [main\(\) method in Java](#)

[↑ TOC](#)

1.5 Data Types

Definition: Java has two categories of data types — **primitive types** (8, built into the language, store raw values) and **reference types** (objects, arrays, strings — store a reference/address to the actual data on the heap).

```
// — 8 Primitive Types —
byte    b  = 127;           // 1 byte,  -128 to 127
short   sh = 32000;         // 2 bytes
int      i  = 100000;        // 4 bytes  (default for integers)
long     l  = 100000000000L; // 8 bytes  (note the 'L' suffix)
float    f  = 3.14f;         // 4 bytes  (note the 'f' suffix)
double   d  = 3.14159;       // 8 bytes  (default for decimals)
char     c  = 'A';           // 2 bytes, Unicode
boolean  flag = true;        // 1 bit (JVM-dependent storage),
                             true/false only

// — Reference Types —
String   s      = "Java";    // object
int[]    arr    = {1, 2, 3};  // array (object)
Object   obj    = new Object(); // any class

// Wrapper classes (autoboxing/unboxing bridges primitives ↔ objects)
Integer  boxedInt = 10;      // autoboxing: primitive automatically wrapped
into an object
int      unboxed  = boxedInt; // unboxing: object automatically converted back
to a primitive
```

Memory Units Quick Reference:

Order	Unit	Abbreviation	Conversion
1	Bit	bit	Smallest unit
2	Byte	B	1 Byte = 8 Bits
3	Kilobyte	KB	1 KB = 1,024 Bytes
4	Megabyte	MB	1 MB = 1,024 KB
5	Gigabyte	GB	1 GB = 1,024 MB
6	Terabyte	TB	1 TB = 1,024 GB
7	Petabyte	PB	1 PB = 1,024 TB
8	Exabyte	EB	1 EB = 1,024 PB
9	Zettabyte	ZB	1 ZB = 1,024 EB
10	Yottabyte	YB	1 YB = 1,024 ZB

Smallest → Largest: Bit → Byte → KB → MB → GB → TB → PB → EB → ZB → YB

Remember: 1 Byte = 8 Bits · 1 KB = 1,024 Bytes · 1 MB = 1,024 KB · 1 GB = 1,024 MB · 1 TB = 1,024 GB · 1 PB = 1,024 TB · 1 EB = 1,024 PB · 1 ZB = 1,024 EB · 1 YB = 1,024 ZB

Type	Size	Default	Range
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	~-2.1B to 2.1B
long	8 bytes	0L	~-9.2×10 ¹⁸ to 9.2×10 ¹⁸
float	4 bytes	0.0f	~±3.4×10 ³⁸ (7 digits precision)
double	8 bytes	0.0d	~±1.8×10 ³⁰⁸ (15 digits precision)
char	2 bytes	'\u0000'	0 to 65,535 (Unicode)
boolean	JVM-dependent	false	true / false

Exam tip: Wrapper classes matter because Java Collections (`ArrayList`, `HashMap`, etc.) can only store *objects*, not primitives — this is *why* autoboxing exists as a language feature.

 **GFG:** [Data Types in Java](#)

[↑ TOC](#)

1.6 Variables & Memory Concepts

Definition: Java has three kinds of variables based on scope and lifetime — **static (class) variables**, **instance variables**, and **local variables** — and each is stored in a different JVM memory area.

```
public class MemoryDemo {
    static int classField = 10;           // static/class variable - Method
    Area/Metaspace, ONE copy shared

    int instanceField = 20;               // instance variable - Heap, one copy
    PER OBJECT

    void demo() {
        int localVar = 30;               // local variable - Stack, exists only
        during method call
        // must be initialized before use - no default value like fields
    }

    public static void main(String[] args) {
        MemoryDemo obj1 = new MemoryDemo(); // obj1 reference on Stack,
        object data on Heap
        MemoryDemo obj2 = new MemoryDemo();

        obj1.instanceField = 99;
        System.out.println(obj2.instanceField); // 20 - separate copy,
        unaffected
    }
}
```

```

        classField = 77;
        System.out.println(MemoryDemo.classField); // 77 - shared by ALL
instances
    }
}

```

JVM Runtime Memory Areas (very common diagram question):

Method Area / Metaspace	→ class metadata, static fields, constant pool
Heap	→ all objects & instance fields (GC managed)
Stack (per thread)	→ local variables, method call frames
PC Register (per thread)	→ address of currently executing instruction
Native Method Stack	→ for native (non-Java) method calls

Explain in words: Each thread gets its own Stack, PC Register, and Native Method Stack (thread-private), whereas the Heap and Method Area are shared across all threads of the JVM — this is exactly why instance/static variables need synchronization in multithreading but local variables don't.

 **GFG:** [JVM Memory Model in Java](#)

[↑ TOC](#)

1.7 Control Statements

```

// if / else if / else
int marks = 75;
if (marks >= 90) System.out.println("A+");
else if (marks >= 75) System.out.println("A");
else System.out.println("B");

// switch (classic + modern arrow form, Java 14+)
int day = 3;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    default: System.out.println("Other");
}

String dayName = switch (day) {
    case 1 -> "Mon";
    case 2 -> "Tue";
    default -> "Other";
};

```

```
// ternary
String result = (marks >= 40) ? "Pass" : "Fail";
```

Exam note on `switch`: Explain that `break` prevents *fall-through* (execution continuing into the next case) in the classic form; the newer arrow (`->`) form does not fall through automatically and can directly return a value, which is why it's used to initialize `dayName` above.

 **GFG:** [Decision Making in Java \(if, switch\)](#)

[↑ TOC](#)

1.8 Looping Constructs

```
// for – used when the number of iterations is known in advance
for (int i = 0; i < 5; i++) System.out.println(i);

// while – condition checked BEFORE the body executes; may run zero times
int i = 0;
while (i < 5) { System.out.println(i); i++; }

// do-while – condition checked AFTER the body; guarantees AT LEAST one
// execution
int j = 0;
do { System.out.println(j); j++; } while (j < 5);

// enhanced for (for-each) – used to iterate over arrays/collections
// without an index
int[] nums = {10, 20, 30};
for (int n : nums) System.out.println(n);

// break / continue with labels – used to control OUTER loops from an inner
// loop
outer:
for (int a = 0; a < 3; a++) {
    for (int b = 0; b < 3; b++) {
        if (b == 1) continue outer;    // skips to the next iteration of the
// OUTER loop
        if (a == 2) break outer;       // exits the OUTER loop entirely
        System.out.println(a + "," + b);
    }
}
```

 **GFG:** [Loops in Java](#)

[↑ TOC](#)

1.9 Method Call Stack & Activation Records

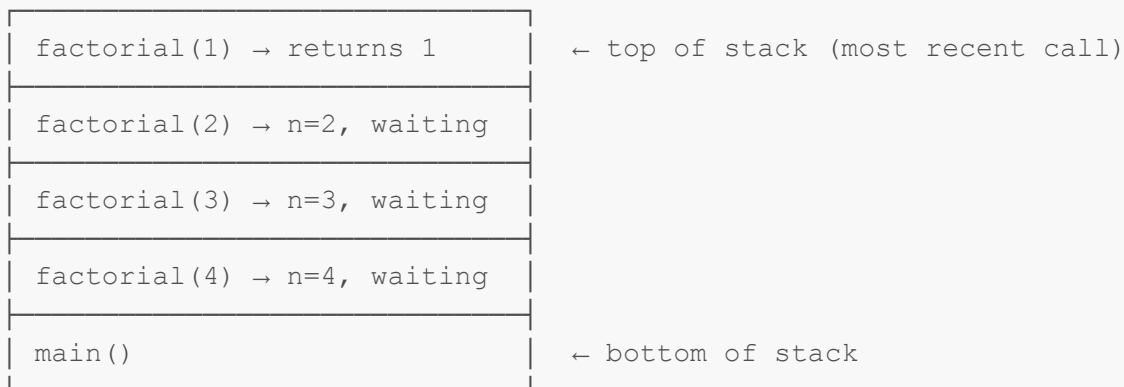
Definition: Every time a method is called, the JVM creates a new **stack frame (Activation Record)** on the calling thread's **call stack**, which stores that call's local data.

```
public class CallStackDemo {
    static int factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);    // each call pushes a NEW activation
record
    }

    public static void main(String[] args) {
        System.out.println(factorial(4));
    }
}
```

Diagram to draw for exam:

Call stack while computing factorial(4):



What an Activation Record stores (list these four for full marks):

1. The method's parameters
2. Its local variables
3. The return address (where control resumes in the caller)
4. The operand stack used for intermediate computation

 **Important exam point:** Excessive recursion depth exhausts the stack, causing a `StackOverflowError` — because each thread's stack has a fixed, limited size.

 **GFG:** [Call Stack / Recursion in Java](#)

[↑ TOC](#)

1.10 Argument Promotion and Casting

Definition: *Type conversion* in Java is either **widening** (implicit, smaller→larger type, safe) or **narrowing** (explicit cast required, larger→smaller type, risk of data loss).

```
// Widening (automatic / implicit) – smaller type → larger type, no data
loss
byte b = 10;
int i = b;           // byte → int: automatic
long l = i;          // int → long: automatic
double d = l;         // long → double: automatic

// Narrowing (explicit cast required) – larger type → smaller type,
possible data loss
double pi = 3.99;
int truncated = (int) pi;    // 3 – explicit cast needed, fraction is lost
long big = 130L;
byte overflowed = (byte) big; // overflow – result wraps around
(unpredictable without care)

// Argument promotion in expressions/method calls:
// byte, short, char are promoted to int automatically in arithmetic
expressions
byte x = 5, y = 10;
int sum = x + y;    // byte+byte promotes to int automatically – CANNOT
assign to byte directly
// byte wrongSum = x + y; // ❌ compile error – needs an explicit cast:
(byte) (x + y)

// Widening also applies when choosing an overloaded method (see 1.11)
void show(long n) { System.out.println("long: " + n); }
show(5);    // int 5 is WIDENED to long automatically – calls show(long)
```

Explain the "why": Widening is always safe because the target type can represent every value of the source type, so Java allows it silently. Narrowing may lose information (e.g., a `double`'s decimal part, or an `int` value too large for a `byte`), so the compiler forces you to acknowledge the risk with an explicit cast.

 **GFG:** [Type Conversion in Java \(Widening/Narrowing\)](#)

[↑ TOC](#)

1.11 Scope of Declaration and Method Overloading

```
public class ScopeDemo {
    int field = 100;           // class-level scope – visible
    throughout the class

    void method1() {
        int local = 10;       // method-level scope – only visible
        inside method1()
        {
            int blockVar = 20; // block scope – only visible inside
            these { }
            System.out.println(local + blockVar);
        }
    }
}
```

```

    }
    // blockVar is NOT accessible here – out of scope
}

// — Method Overloading – same name, different parameter list —
void print(int i)           { System.out.println("int: " + i); }
void print(double d)        { System.out.println("double: " + d); }
void print(String s)        { System.out.println("String: " + s); }
void print(int a, int b)    { System.out.println("two ints: " + (a
+ b)); }

public static void main(String[] args) {
    ScopeDemo obj = new ScopeDemo();
    obj.print(5);
    obj.print(3.14);
    obj.print("Hello");
    obj.print(2, 3);
}
}

```

Definition of Method Overloading: Defining multiple methods in the same class with the *same name* but a *different parameter list* (different number, type, or order of parameters). Return type alone is **not** sufficient to overload.

Overload resolution order (compiler decides at compile-time — a favourite exam question):

1. Exact type match
2. Widening primitive conversion
3. Autoboxing/unboxing
4. Varargs

The compiler tries these steps *in this order* and stops at the first one that produces a matching method.

 **GFG:** [Method Overloading in Java](#)

[↑ TOC](#)

1.12 String Handling

String Constructors

```

String s1 = "Java";           // string literal – stored
                              // in the String Pool
String s2 = new String("Java"); // new object on heap –
NOT pooled
String s3 = new String(new char[]{'J','a','v','a'});
String s4 = new String(s1);    // copy constructor style
byte[] bytes = {72, 105};
String s5 = new String(bytes); // "Hi" – from a byte
array

```

```
System.out.println(s1 == s2);           // false - different objects
System.out.println(s1.equals(s2));      // true - same content
```

Explain the String Pool concept (important): The **String Constant Pool** is a special memory area inside the Heap where Java stores string literals. When you write `String s1 = "Java"`, the JVM checks the pool first — if `"Java"` already exists, `s1` simply points to that existing object (saving memory); if not, a new one is added. Using `new String(...)` bypasses the pool and always creates a fresh object on the heap, which is why `s1 == s2` is `false` (different memory addresses) even though `.equals()` returns `true` (same content).

String Operators

```
String a = "Hello";
String b = "World";
String concatenated = a + " " + b;           // '+' is Java's ONLY overloaded
operator, for String concatenation
concatenated += "!";                         // '+=' also works for
concatenation
System.out.println(concatenated);            // Hello World!

// String is IMMUTABLE - every "modification" creates a NEW String object
String x = "Hi";
x.concat(" there"); // return value discarded - x is unchanged!
System.out.println(x); // still "Hi"
```

Why String is immutable (common theory question, 2–3 marks):

- **Security** — Strings are widely used for things like file paths, network URLs, and class names; immutability prevents them from being changed unexpectedly after being passed around.
- **String Pool efficiency** — sharing pooled literals is only safe if strings can never change.
- **Thread-safety** — an immutable object can be freely shared between threads without synchronization.
- **Caching of hashcode** — since content never changes, the hash code can be computed once and cached (useful in `HashMap` keys).

Character Extraction

```
String str = "Java Programming";

char ch = str.charAt(0);           // 'J'
char[] chars = str.toCharArray(); // full char array
str.getChars(0, 4, chars, 0);      // copies a range into a
destination array
byte[] bytes = str.getBytes();      // byte array (platform default
encoding)

System.out.println("Length: " + str.length());
System.out.println("Substring: " + str.substring(5)); //
```

```
"Programming"
System.out.println("Substring: " + str.substring(0, 4));    // "Java"
```

String Comparison

```
String s1 = "apple";
String s2 = "Apple";

System.out.println(s1.equals(s2));           // false - case-sensitive
System.out.println(s1.equalsIgnoreCase(s2)); // true
System.out.println(s1.compareTo(s2));        // >0 (lowercase 'a' >
uppercase 'A' in Unicode)
System.out.println(s1.compareToIgnoreCase(s2)); // 0
System.out.println(s1 == s2);                 // false (different content
anyway)
System.out.println(s1.regionMatches(0, s2, 0, 3)); // false - 'a' vs 'A'
System.out.println(s1.startsWith("app"));      // true
System.out.println(s1.endsWith("le"));         // true
System.out.println(s1.contains("pl"));         // true
```

Rule to remember: Always use `.equals()` (or `.equalsIgnoreCase()`) to compare string *content*; `==` compares *references* (memory addresses) and should only be used when you specifically want to check object identity.

StringBuffer / StringBuilder (Mutable Strings)

```
StringBuffer sb = new StringBuffer("Hello");    // thread-safe
(synchronized methods)
StringBuilder sbd = new StringBuilder("Hello"); // NOT thread-safe, but
FASTER - prefer this for single-threaded code

sb.append(" World");           // Hello World
sb.insert(5, ",");             // Hello, World
sb.reverse();                  // dlroW ,olleH
sb.reverse();                  // back to Hello, World
sb.replace(0, 5, "Hi");        // Hi, World
sb.delete(0, 3);               // World
sb.setCharAt(0, 'w');          // world
System.out.println(sb);
System.out.println("Capacity: " + sb.capacity()); // default 16 + initial
string length
```

String	StringBuffer	StringBuilder
Immutable	Mutable	Mutable

String	StringBuffer	StringBuilder
Thread-safe (by immutability)	Thread-safe (synchronized methods)	NOT thread-safe
Slow for repeated modification (creates new objects each time)	Slower (synchronization overhead)	Fastest for single-threaded use

Exam tip: This comparison table (String vs StringBuffer vs StringBuilder) is asked almost every year — memorize it exactly.

 **GFG:** [String vs StringBuffer vs StringBuilder](#)

[↑ TOC](#)

1.13 Arrays

Definition: An array is a fixed-size, ordered collection of elements of the *same data type*, stored in contiguous memory, and treated as an **object** in Java (unlike C/C++).

Declaring and Creating Arrays

```
int[] arr1;           // declaration (preferred style)
int arr2[];           // declaration (C-style, also legal)

arr1 = new int[5];     // creation – default-initialized to 0
int[] arr3 = new int[]{1, 2, 3, 4, 5}; // creation + initialization
int[] arr4 = {1, 2, 3, 4, 5}; // shorthand – only at declaration
```

Enhanced for Statement (for-each)

```
int[] nums = {10, 20, 30, 40};
for (int n : nums) {
    System.out.println(n);
}
// Works with any array or Iterable (List, Set, etc.)
// Read-only view – modifying 'n' does NOT change the array element
```

Passing Arrays to Methods

```
public class ArrayPassDemo {
    static void modify(int[] arr) {
        arr[0] = 999; // arrays are objects – passed by reference (the
        // reference itself is copied)
    }
}
```

```

    public static void main(String[] args) {
        int[] data = {1, 2, 3};
        modify(data);
        System.out.println(data[0]); // 999 - the original array WAS
modified
    }
}

```

Important theory point: Java is technically **"pass by value" always** — but for objects (including arrays), the *value* being passed is a copy of the reference (memory address), so changes made *through* that reference inside the method affect the original object. This is a subtle and commonly-asked conceptual question.

Multidimensional Arrays

```

int[][] matrix = new int[3][3]; // 3x3, all zeros
int[][] initialized = {{1,2,3}, {4,5,6}, {7,8,9}};

for (int[] row : initialized) {
    for (int val : row) System.out.print(val + " ");
    System.out.println();
}

// Jagged arrays - Java arrays-of-arrays can have DIFFERENT row lengths
int[][] jagged = new int[3][];
jagged[0] = new int[]{1};
jagged[1] = new int[]{1, 2};
jagged[2] = new int[]{1, 2, 3};

```

Explain: Java doesn't have true multidimensional arrays like C — a 2D array is actually an *array of arrays*. This is exactly why "jagged" (ragged) arrays, where each row has a different length, are possible.

Variable-Length Argument Lists (Varargs)

```

public class VarargsDemo {
    static int sum(int... numbers) { // '...' means "zero or more ints",
accessible as an array
        int total = 0;
        for (int n : numbers) total += n;
        return total;
    }

    // Varargs must be the LAST parameter
    static void describe(String name, int... scores) {
        System.out.println(name + " has " + scores.length + " scores");
    }

    public static void main(String[] args) {
        System.out.println(sum()); // 0
        System.out.println(sum(1, 2, 3)); // 6
    }
}

```

```
        System.out.println(sum(1, 2, 3, 4, 5)); // 15
        describe("Rohit", 90, 85, 78);
    }
}
```

Rule to state: A method can have **at most one varargs parameter**, and it **must be the last parameter** in the list, because the compiler needs to unambiguously determine where the fixed parameters end and the variable-length ones begin.

Using Command-line Arguments

```
public class CmdArgsDemo {
    public static void main(String[] args) {
        System.out.println("Number of args: " + args.length);
        for (String arg : args) {
            System.out.println("Arg: " + arg);
        }
    }
}
```

```
javac CmdArgsDemo.java
java CmdArgsDemo hello world 123
# Output:
# Number of args: 3
# Arg: hello
# Arg: world
# Arg: 123
```

 [GFG: Arrays in Java](#)

[↑ TOC](#)

UNIT-II

2.1 Inheritance: Extending Classes

Definition: Inheritance is an OOP mechanism where a subclass (child class) acquires the fields and methods of a superclass (parent class) using the `extends` keyword, promoting code reuse and establishing an "is-a" relationship.

```
class Vehicle {
    protected String brand;
    protected int speed;
}
```

```

    Vehicle(String brand, int speed) { this.brand = brand; this.speed =
speed; }

    void display() { System.out.println(brand + " @ " + speed + " km/h"); }
}

class Car extends Vehicle {           // 'extends' - single inheritance
only
    int doors;

    Car(String brand, int speed, int doors) {
        super(brand, speed);           // must be FIRST statement - calls
parent constructor
        this.doors = doors;
    }

    @Override
    void display() {
        super.display();               // call parent's version explicitly
        System.out.println("Doors: " + doors);
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Car c = new Car("Tata", 180, 4);
        c.display();

        Vehicle v = c;                 // upcasting - always safe
        v.display();                   // still calls Car's overridden
display() - dynamic dispatch

        // Downcasting - needs explicit cast + instanceof check
        if (v instanceof Car car2) {
            System.out.println("Downcast successful, doors: " +
car2.doors);
        }
    }
}

```

Key rules (list all of these for a "rules of inheritance" question):

- Java supports **single inheritance for classes** (a class can extend only one superclass) but a class can implement multiple interfaces.
- `super` is used to call the parent class's constructor (must be the first statement in the child constructor) or to explicitly call an overridden parent method.
- Every class implicitly extends `Object` if no other superclass is specified — so every Java class inherits methods like `toString()`, `equals()`, `hashCode()`.
- A `final` class cannot be extended; a `final` method cannot be overridden.

- **Upcasting** (child → parent reference) is always safe and implicit; **downcasting** (parent → child reference) requires an explicit cast and should be guarded with `instanceof` to avoid a `ClassCastException`.
- Because of **dynamic method dispatch**, even when `v` is declared as type `Vehicle`, calling `v.display()` executes `Car`'s overridden version — the JVM decides which method to run based on the *actual object type at runtime*, not the reference type.

 **GFG:** [Inheritance in Java](#)

[↑ TOC](#)

2.2 Packages and Interfaces

Definition: A package is a namespace/folder mechanism used to group related classes and interfaces together, avoid naming conflicts, and control access.

Defining a Package

```
// File: com/rohit/utils/MathHelper.java
package com.rohit.utils;

public class MathHelper {
    public static int square(int x) { return x * x; }
}
```

Rule: The directory structure **must exactly mirror** the package name:

`com/rohit/utils/MathHelper.java`.

Understanding CLASSPATH

CLASSPATH tells the JVM/compiler where to look for `.class` files and libraries.

Set via:

- Environment variable: `export CLASSPATH=/path/to/classes:/path/to/lib.jar`
- Command-line flag: `java -cp /path/to/classes MainClass`
- Default: current directory (`.`) if CLASSPATH is unset

Example:

```
javac -d build src/com/rohit/utils/MathHelper.java # -d places compiled
classes per package structure
java -cp build com.rohit.utils.Main
```

Access Protection

Modifier	Same Class	Same Package	Subclass (diff. package)	Other Package
<code>private</code>	✓	×	×	×
<code>(default)</code>	✓	✓	×	×
<code>protected</code>	✓	✓	✓	×
<code>public</code>	✓	✓	✓	✓

Exam tip: This access-modifier table is asked frequently as a direct question — memorize it exactly as a grid.

Importing Packages

```
import java.util.List;           // import a single class
import java.util.*;             // import all classes in a package (not
sub-packages)
import static java.lang.Math.PI; // static import — use PI directly
instead of Math.PI
import com.rohit.utils.MathHelper;

public class ImportDemo {
    public static void main(String[] args) {
        System.out.println(MathHelper.square(5));
        System.out.println(PI);
        List<Integer> nums = new ArrayList<>();
    }
}
```

Creating Your Own Packages — Full Example

```
// File: shapes/Shape.java
package shapes;

public interface Shape {
    double area();
}

// File: shapes/Circle.java
package shapes;

public class Circle implements Shape {
    private double radius;
    public Circle(double radius) { this.radius = radius; }
    public double area() { return Math.PI * radius * radius; }
}

// File: Main.java (default/no package, or a different package)
import shapes.Shape;
import shapes.Circle;
```

```
public class Main {  
    public static void main(String[] args) {  
        Shape s = new Circle(5);  
        System.out.println("Area: " + s.area());  
    }  
}
```

```
javac -d . shapes/Shape.java shapes/Circle.java Main.java  
java Main
```

Interface vs Abstract class (common comparison question — write this even if not directly asked about interfaces):

- An **interface** declares only method signatures (abstract by default) — before Java 8 it could have no implementation at all; a class **implements** an interface and must provide bodies for all its methods, and a class can implement *multiple* interfaces (achieving a form of multiple inheritance).
- An **abstract class** can have both abstract and concrete (fully implemented) methods, and a class can **extend** only *one* abstract class.

 **GFG:** [Packages and Interfaces in Java](#)

[↑ TOC](#)

2.3 Exception Handling

Definition: Exception handling is a mechanism to detect and handle runtime errors so that the normal flow of the program can be maintained, using the keywords **try**, **catch**, **finally**, **throw**, and **throws**.

Introduction & Keywords

```
try {
    // risky code
} catch (SpecificException e) {
    // handle it
} catch (Exception e) {
    // catch-all fallback
} finally {
    // ALWAYS runs - cleanup code (closing files, connections, etc.)
}

throw new IllegalArgumentException("bad input"); // manually raise an
exception

void riskyMethod() throws java.io.IOException { // declare a CHECKED
exception the method may throw
    // ...
}
```

Meaning of each keyword (write this as a list for definition-type questions):

- **try** — block of code to be monitored for exceptions.
- **catch** — block that handles a specific exception type thrown from the **try** block.
- **finally** — block that always executes, whether or not an exception occurred (used for cleanup, e.g., closing files/connections).
- **throw** — used to explicitly raise/throw an exception object.
- **throws** — used in a method signature to declare which checked exceptions that method might propagate to its caller.

Overview & When to Use

```
public class ExceptionDemo {
    static int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Cannot divide by zero");
        return a / b;
    }

    public static void main(String[] args) {
        try {
            System.out.println(divide(10, 2));
            System.out.println(divide(5, 0)); // throws here
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Cleanup always runs");
        }

        // Custom exception
        try {
            validateAge(15);
        }
    }
}
```

```

        } catch (InvalidAgeException e) {
            System.out.println("Custom exception: " + e.getMessage());
        }
    }

    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) throw new InvalidAgeException("Must be 18+");
    }
}

class InvalidAgeException extends Exception {    // checked — extends
Exception, not RuntimeException
    public InvalidAgeException(String msg) { super(msg); }
}

```

Checked vs Unchecked exceptions (frequently asked, memorize this comparison):

Checked Exception	Unchecked (Runtime) Exception
Checked by the compiler at <i>compile time</i>	Not checked at compile time, occurs at <i>runtime</i>
Must be either caught (<i>try-catch</i>) or declared (<i>throws</i>)	No such requirement
Extends <i>Exception</i> (but not <i>RuntimeException</i>)	Extends <i>RuntimeException</i>
Represents recoverable conditions the caller should anticipate — e.g. <i>IOException</i> , <i>SQLException</i>	Represents programming errors — e.g. <i>NullPointerException</i> , <i>ArithmeticException</i> , <i>ArrayIndexOutOfBoundsException</i>

When to use exceptions (theory point, write this precisely): Exceptions should be used only for *exceptional*, unexpected conditions — such as a missing file, a failed network call, or invalid input — and never as a substitute for normal control flow (e.g., you should not use an exception just to break out of a loop).

 **GFG:** [Exception Handling in Java](#)

[↑ TOC](#)

2.4 Multithreading

Definition: A thread is the smallest unit of CPU execution within a process; multithreading is the ability of a program to execute multiple threads concurrently, with all threads of a process sharing the same memory space (unlike separate processes, which have independent memory).

The Java Thread Model

```

// Method 1: extending Thread
class MyThread extends Thread {

```

```

        public void run() {
            for (int i = 1; i <= 3; i++) {
                System.out.println(getName() + ": " + i);
            }
        }
    }

// Method 2: implementing Runnable (preferred — allows extending another
// class too)
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + ": " +
i);
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        MyThread t1 = new MyThread();
        t1.start();                                // NEVER call run() directly —
start() creates a new OS thread

        Thread t2 = new Thread(new MyRunnable());
        t2.start();

        // Lambda (Runnable is a functional interface)
        Thread t3 = new Thread(() -> System.out.println("Lambda thread
running"));
        t3.start();

        t1.join();    // wait for t1 to finish before continuing
        System.out.println("All threads started");
    }
}

```

Two ways to create a thread — compare them (common question):

1. **Extending Thread** — simple, but since Java doesn't allow multiple inheritance, your class can no longer extend anything else.
2. **Implementing Runnable** — the preferred approach, because your class remains free to extend another class, and it separates the *task* (what to run) from the *thread mechanism* (how it runs).

Critical rule: Always call `.start()`, never `.run()` directly — calling `run()` directly just executes it as a normal method call on the current thread (no new thread is created); only `start()` asks the JVM to create a new call stack and actually run `run()` concurrently.

Thread Priorities

```
Thread t = new Thread(() -> System.out.println("Running"));
t.setPriority(Thread.MAX_PRIORITY);    // 10
t.setPriority(Thread.MIN_PRIORITY);    // 1
t.setPriority(Thread.NORM_PRIORITY);   // 5 (default)
// Priority is a HINT to the scheduler – not a guarantee, behavior is
// JVM/OS dependent
```

Thread Life Cycle

```
NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED
                    ↑
                (waiting for lock, join(), sleep(), etc.)
```

State	Meaning
NEW	Thread object created, <code>start()</code> not yet called
RUNNABLE	Executing or ready to execute (scheduler decides)
BLOCKED	Waiting to acquire a monitor lock (<code>synchronized</code>)
WAITING	Waiting indefinitely for another thread (<code>wait()</code> , <code>join()</code>)
TIMED_WAITING	Waiting for a specified time (<code>sleep(ms)</code> , <code>join(ms)</code>)
TERMINATED	<code>run()</code> method has completed

Exam tip: Draw this life-cycle diagram *and* the table together — diagram questions on thread states are very common (5+ marks).

Thread Synchronization

Definition: Synchronization is a mechanism that controls access of multiple threads to a shared resource, preventing a **race condition** (an inconsistent result caused by threads interleaving their operations unpredictably on shared data).

```
class Counter {
    private int count = 0;

    // synchronized method – only ONE thread can execute this at a time per
    // object instance
    public synchronized void increment() {
        count++;
    }

    public int getCount() { return count; }
}

public class SyncDemo {
```

```

    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final count: " + counter.getCount()); // 2000
        - safe due to synchronization
        // Without 'synchronized', race conditions could give an
        inconsistent/lower count!
    }
}

// synchronized block - locks only the critical section, not the whole
method
void transfer(Account from, Account to, double amount) {
    synchronized (from) {
        synchronized (to) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}
}

```

Explain the mechanism: Every object in Java has an intrinsic lock (monitor). When a thread enters a `synchronized` method/block, it acquires the lock on that object; any other thread trying to enter a `synchronized` section on the *same object* must wait until the lock is released. A `synchronized block` is generally preferred over a `synchronized method` because it locks only the critical section (the minimum necessary code), improving performance by allowing more concurrent execution elsewhere.

 [GFG: Multithreading in Java](#)

[↑ TOC](#)

2.5 Applets

 **Note:** Applets are **deprecated** (Java 9) and **removed** (Java 17+) — browsers dropped Java plugin support years ago. Covered here for syllabus/legacy-exam purposes only; do not use for new projects.

Applet Basics & Architecture

Definition: An applet is a small Java program that runs inside a web browser (or an applet viewer), embedded in an HTML page.

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, Applet World!", 20, 20);
    }
}
```

```
<!-- HelloApplet.html -->
<applet code="HelloApplet.class" width="300" height="100"></applet>
```

Architecture (explain in exam): An applet executes inside the browser's own JVM in a restricted **sandbox environment** — it cannot access the local file system, cannot open arbitrary network connections (except back to its own originating server), and cannot run native code, unlike a standalone Java application which has full system access.

Applet Life Cycle Methods

```
init()      → called ONCE, when the applet is first loaded (setup)
start()     → called after init(), and every time the applet's page is
revisited
paint()     → called whenever the applet needs to redraw itself
stop()      → called when the user navigates away from the page
destroy()   → called ONCE, when the browser shuts down / applet is unloaded
```

```
import java.applet.Applet;
import java.awt.Graphics;

public class LifecycleApplet extends Applet {
    public void init()      { System.out.println("init: setup resources"); }
    public void start()     { System.out.println("start: begin execution"); }
    public void paint(Graphics g) { g.drawString("Running...", 10, 10); }
    public void stop()      { System.out.println("stop: pause execution"); }
    public void destroy()   { System.out.println("destroy: release
resources"); }
}
```

Exam tip: Note the order clearly: `init()` → `start()` → `paint()` on first load; `stop()` → `start()` → `paint()` if the user leaves and returns to the page; `stop()` → `destroy()` when the browser closes the page permanently.

2.6 Database Connectivity: JDBC

Definition: JDBC (Java Database Connectivity) is a Java API that provides a standard, vendor-independent way for Java applications to connect to and interact with relational databases.

The Design of JDBC



Four Driver Types (explain briefly — often a direct question):

- **Type 1** — JDBC-ODBC Bridge driver (translates JDBC calls into ODBC calls); removed from modern JDK, obsolete.
- **Type 2** — Native-API driver; converts JDBC calls into vendor-specific native (C/C++) library calls; partly Java.
- **Type 3** — Network Protocol driver; sends calls to a middleware server, which translates them for the database; fully Java, but needs a middle server.
- **Type 4** — **Pure Java (thin) driver**; talks directly to the database using its native network protocol — no native code, no middleware. **This is the modern standard** (e.g., `mysql-connector-j`).

Typical Uses of JDBC

```
import java.sql.*;

public class JdbcDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/schooldb";
        String user = "root";
        String password = "password";

        // try-with-resources - auto-closes Connection, Statement,
        // ResultSet
        try (Connection conn = DriverManager.getConnection(url, user,
            password);
```

```

        Statement stmt = conn.createStatement() {

            // SELECT
            ResultSet rs = stmt.executeQuery("SELECT id, name, cgpa FROM
students");
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " | " +
rs.getString("name") + " | " + rs.getFloat("cgpa"));
            }

            // INSERT / UPDATE / DELETE via PreparedStatement (prevents SQL
injection)
            try (PreparedStatement ps = conn.prepareStatement(
                "INSERT INTO students (name, cgpa) VALUES (?, ?)")) {
                ps.setString(1, "Rohit");
                ps.setFloat(2, 9.2f);
                ps.executeUpdate();
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

Steps to connect to a database via JDBC (standard 5-step answer — memorize this sequence):

1. Load/register the JDBC driver (modern drivers auto-register via `ServiceLoader`, so this is often implicit today).
2. Establish a connection using `DriverManager.getConnection(url, user, password)`.
3. Create a `Statement/PreparedStatement/CallableStatement`.
4. Execute the query (`executeQuery()` for SELECT, `executeUpdate()` for INSERT/UPDATE/DELETE).
5. Process the `ResultSet` and finally close the connection (or use try-with-resources to do this automatically).

Common uses: generating reports from queried data, inserting/updating transactional records, connection pooling (via `DataSource`), calling stored procedures (`CallableStatement`), batch updates.

 **GFG: JDBC in Java**

[↑ TOC](#)

UNIT-III: HTTP, Web Servers & Servlets

3.1 Introduction to HTTP, Web Servers, and Application Servers

Definition: HTTP (HyperText Transfer Protocol) is a stateless, text-based request/response protocol used for communication between a client (browser) and a server over the web.

```
Client (Browser) --request (GET/POST/PUT/DELETE)--> Server --response
(status + body)--> Client
```

"Stateless" — explain this clearly (commonly asked): Each HTTP request is independent — the server does not automatically remember any information from a client's previous request. This is *why* Java web technology needs separate session-management mechanisms (cookies, `HttpSession`, URL rewriting — see [Section 3.5](#)) to maintain state across multiple requests from the same user.

Web Server vs Application Server (very frequently asked comparison):

Web Server	Application Server
Serves only static content (HTML, CSS, JS, images)	Runs dynamic Java code (Servlets, JSP, EJB)
Examples: Apache HTTPD, Nginx	Examples: Apache Tomcat, WildFly, GlassFish
Cannot execute business logic	Can execute business logic, connect to databases, manage transactions

(Note: Tomcat is technically a lightweight "Servlet/JSP container", sitting between a plain web server and a full application server like WildFly/GlassFish which also supports EJB.)

Installation of Application Servers (Apache Tomcat example)

```
# 1. Download Tomcat from tomcat.apache.org
# 2. Extract it
tar -xzf apache-tomcat-10.x.tar.gz

# 3. Set JAVA_HOME (Tomcat needs a JDK)
export JAVA_HOME=/path/to/jdk

# 4. Start the server
cd apache-tomcat-10.x/bin
./startup.sh          # Linux/Mac
startup.bat           # Windows

# 5. Access it
# http://localhost:8080

# 6. Stop the server
./shutdown.sh
```

Config Files

```
apache-tomcat/
├── conf/
```

```

|   |   ├── server.xml      → ports, connectors, hosts configuration
|   |   ├── web.xml         → DEFAULT deployment descriptor (applies to ALL
apps)
|   |   ├── context.xml     → per-application context settings
|   |   └── tomcat-users.xml → manager/admin credentials
|   ├── webapps/            → deployed applications go here (as .war files or
exploded folders)
|   ├── lib/                → shared JAR libraries
|   └── logs/               → server & application logs

```

web.xml (Deployment Descriptor)

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         version="5.0">

    <servlet>
        <servlet-name>HelloServlet</servlet-name>
        <servlet-class>com.rohit.HelloServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>HelloServlet</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>

    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>

    <session-config>
        <session-timeout>30</session-timeout> <!-- minutes -->
    </session-config>
</web-app>

```

 **GFG:** [Web Server vs Application Server](#)

[↑ TOC](#)

3.2 Java Servlets

What is a Servlet?

Definition: A Servlet is a Java class that runs inside a **Servlet container** (e.g., Tomcat) and dynamically handles HTTP requests and generates responses — it is Java EE's replacement for older CGI scripts, and is much faster because it is loaded once and reused across many requests (each handled on its own thread), rather than being spawned as a new OS process per request like CGI.

Servlet Development Process

1. Write a class extending `HttpServlet` (or implementing `Servlet`)
2. Override `doGet()/doPost()` etc.
3. Compile against the `Servlet API JAR` (provided by the container)
4. Package into a `WAR` (Web ARchive) with correct directory structure
5. Configure via `web.xml` OR `@WebServlet` annotation
6. Deploy the `.war` to the container's `webapps/` folder
7. Container loads, initializes, and routes requests to it

Deployment Descriptor (web.xml) vs Annotations

```
// Modern approach – annotation-based (Servlet 3.0+), no web.xml entry
// needed
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<h1>Hello from Servlet!</h1>");
    }
}
```

Generic Servlet vs HTTP Servlet

```
import jakarta.servlet.*;
import java.io.*;

// GenericServlet – protocol-independent (rarely used directly; you
// override service())
public class MyGenericServlet extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res) throws
IOException {
        res.setContentType("text/plain");
        res.getWriter().println("Handled by GenericServlet");
    }
}

// HttpServlet – HTTP-specific (the one you use 99% of the time)
// provides doGet(), doPost(), doPut(), doDelete(), doHead(), etc.
```

Comparison table (a favourite short-answer question):

GenericServlet	HttpServlet
Protocol-independent (works with any protocol, not just HTTP)	HTTP-specific
You must override the single <code>service()</code> method yourself	Internally implements <code>service()</code> , which dispatches to <code>doGet()</code> / <code>doPost()</code> /etc. based on the HTTP method
Rarely used directly in real projects	Used in almost all real-world web applications

Lifecycle of a Servlet

Very important — this is asked almost every exam.

```

init()      → called ONCE when the servlet is first loaded (config setup,
resource acquisition)
service()   → called for EVERY request; internally dispatches to
doGet()/doPost()/etc.
              based on the HTTP method
destroy()   → called ONCE when the container unloads/shuts down the servlet

```

```

import jakarta.servlet.http.*;
import jakarta.servlet.*;
import java.io.*;

public class LifecycleServlet extends HttpServlet {
    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        System.out.println("init(): Servlet loaded");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws IOException {
        resp.getWriter().println("service()/doGet(): handling request");
    }

    @Override
    public void destroy() {
        System.out.println("destroy(): Servlet unloaded");
    }
}

```

Explain why this design is efficient: Because `init()` runs only *once* (not per-request), expensive setup like opening a database connection pool can be done a single time and reused across thousands of subsequent requests handled by `service()` — this is the core reason servlets outperform the old CGI model, where every single request spawned a brand-new process.

GFG: Servlet Life Cycle in Java

[↑ TOC](#)

3.3 Servlet Packages, Classes, Interfaces & Methods

```
jakarta.servlet           → Servlet, ServletConfig, ServletContext,
                           ServletRequest, ServletResponse (protocol-agnostic)
jakarta.servlet.http      → HttpServlet, HttpServletRequest,
                           HttpServletResponse, HttpSession, Cookie
```

Key interfaces:

```
- Servlet           → init(), service(), destroy(), getServletConfig(),
getServletInfo()
- ServletRequest    → getParameter(), getAttribute(), getInputStream()
- ServletResponse    → getWriter(), setContentType(), getOutputStream()
```

Key classes:

```
- GenericServlet     → abstract, protocol-independent base implementing
Servlet
- HttpServlet        → abstract, extends GenericServlet, HTTP-specific
(doGet/doPost/...)
- HttpServletRequest/Response → HTTP-specific request/response
- HttpSession        → per-user server-side session storage
- Cookie             → small client-side key-value data
```

Difference between `ServletConfig` and `ServletContext` (commonly confused, good to clarify):

- `ServletConfig` — holds init parameters **specific to one servlet** (defined in that servlet's `<init-param>` in `web.xml`); scope is that single servlet.
- `ServletContext` — holds parameters and shared attributes **for the entire web application** (`<context-param>`); scope is application-wide, shared by all servlets/JSPs in that app.

GFG: ServletConfig vs ServletContext

[↑ TOC](#)

3.4 Handling Forms with a Servlet

```
<!-- form.html -->
<form action="register" method="post">
  Name: <input type="text" name="username" /> Email:
  <input type="email" name="email" />
  <input type="submit" value="Register" />
</form>
```

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/register")
public class RegisterServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws IOException {
        String username = req.getParameter("username");    // reads form
        // field by 'name' attribute
        String email = req.getParameter("email");

        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<h2>Registered: " + username + " (" + email + ")
        </h2>");
    }
}
```

Explain GET vs POST here (common follow-up): **GET** appends form data to the URL as a query string (visible, limited length, cacheable) — used for `doGet()`; **POST** sends form data in the request body (hidden from the URL, no size limit, not cached by default) — used for `doPost()`. The `method` attribute in the `<form>` tag decides which one the browser uses, and correspondingly which servlet method handles it.

 **GFG:** [Handling Form Data in Servlet](#)

[↑ TOC](#)

3.5 Session Handling — Various Methods

Definition: Since HTTP is stateless, *session tracking* refers to techniques used to maintain a user's state/data across multiple requests. Java Servlets provide four such methods.

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/session-demo")
public class SessionServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
    throws IOException {
        // Method 1: HttpSession (most common — server-side storage,
        // session ID via cookie/URL)
        HttpSession session = req.getSession();    // creates one if it
        // doesn't exist
        Integer visits = (Integer) session.getAttribute("visits");
        visits = (visits == null) ? 1 : visits + 1;
    }
}
```

```

        session.setAttribute("visits", visits);

        // Method 2: Cookies (client-side storage)
        Cookie cookie = new Cookie("lastVisit",
String.valueOf(System.currentTimeMillis()));
        cookie.setMaxAge(60 * 60 * 24);    // 1 day
        resp.addCookie(cookie);

        // Method 3: URL Rewriting (fallback when cookies are disabled)
        String encodedUrl = resp.encodeURL("/session-demo");

        // Method 4: Hidden form fields – pass session data as invisible
        <input type="hidden">

        resp.getWriter().println("Visits: " + visits + " | Encoded URL: " +
encodedUrl);
    }
}

```

All four session-tracking techniques (list format for a direct exam question):

1. **HttpSession** — server stores data against a unique session ID; the ID itself is usually transmitted to the client via a cookie (**JSESSIONID**). Most widely used and most powerful (can store any object, not just strings).
2. **Cookies** — small pieces of key-value data stored *on the client's browser* and sent back with every subsequent request to the same domain. Limited size, and users can disable cookies.
3. **URL Rewriting** — the session ID is appended directly to every URL as a query parameter, used as a fallback when the client has disabled cookies.
4. **Hidden form fields** — session data is embedded as invisible fields inside forms and resubmitted with every form post — only works for form-driven navigation, not general page-to-page browsing.

 **GFG:** [Session Tracking in Servlet](#)

[↑ TOC](#)

3.6 Elements of Deployment Descriptors

```

<web-app>
    <servlet>...</servlet>                <!-- register a servlet class -
->
    <servlet-mapping>...</servlet-mapping> <!-- map a URL pattern to a
servlet -->
    <filter>...</filter>                 <!-- register a Filter
(pre/post-processing) -->
    <filter-mapping>...</filter-mapping>
    <listener>...</listener>             <!-- lifecycle event listeners
-->
    <context-param>...</context-param>    <!-- application-wide init
parameters -->

```

```
<welcome-file-list>...</welcome-file-list> <!-- default page
(index.html, etc.) -->
<error-page>...</error-page> <!-- custom error pages per
status code/exception -->
<session-config>...</session-config> <!-- session timeout settings -
->
<security-constraint>...</security-constraint> <!-- URL-based access
control -->
</web-app>
```

Definition to open with: The Deployment Descriptor ([web.xml](#)) is an XML configuration file that tells the Servlet container how to deploy and manage a web application — which servlets exist, what URLs map to them, session timeout, security rules, etc. — without requiring changes to the Java source code.

 **GFG:** [web.xml Deployment Descriptor](#)

[↑ TOC](#)

UNIT-IV: JSP & Database-Backed Web Apps

4.1 JSP Basics

Definition: JSP (JavaServer Pages) is a technology that allows embedding Java code directly inside HTML pages; the container internally translates each JSP page into a Servlet the first time it is requested, so **a JSP is really just a Servlet written in a more HTML-friendly way.**

JSP Lifecycle

1. Translation → JSP file (.jsp) is translated into a Servlet's Java source (_jsp.java)
2. Compilation → that Java source is compiled into a .class file
3. `jspInit()` → called once, like a servlet's `init()`
4. `_jspService()` → called for every request (auto-generated, handles `doGet/doPost`)
5. `jspDestroy()` → called once, like a servlet's `destroy()`

Explain why JSP exists alongside Servlets (common "why JSP" question): Writing HTML output using `out.println()` inside a servlet becomes extremely messy for large pages. JSP flips this around — you write mostly HTML, with small chunks of embedded Java where dynamic content is needed — which is why JSP is preferred for the **View** layer, while Servlets are preferred for the **Controller** (see MVC, [Section 5.1](#)).

Directives

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ page import="java.util.*, java.sql.*" %>
```

```
<%@ include file="header.jsp" %>
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

Scripting Elements

```
<%-- comment (not sent to client) --%>

<% // scriptlet - raw Java code
    int x = 10;
    int y = 20;
%>

<%= x + y %>    <!-- expression - printed directly into the HTML output -->

<%! // declaration - declares a field/method at the servlet class level
    int counter = 0;
    int increment() { return ++counter; }
%>
```

Three scripting elements compared (good direct-answer table):

Element	Syntax	Purpose
Scriptlet	<code><% ... %></code>	Embed a block of plain Java code (statements)
Expression	<code><%= ... %></code>	Evaluate an expression and print its result directly into the output
Declaration	<code><%! ... %></code>	Declare a field or method at the class level of the generated servlet

Standard Actions

```
<jsp:useBean id="student" class="com.rohit.Student" scope="session"/>
<jsp:setProperty name="student" property="name" value="Rohit"/>
<jsp:getProperty name="student" property="name"/>
<jsp:include page="footer.jsp"/>
<jsp:forward page="nextPage.jsp"/>
```

Implicit Objects

Definition: Implicit objects are objects automatically made available inside every JSP page by the container, without needing to be declared — because they're all pre-defined as local variables inside the auto-generated `_jspService()` method.

Object	Type	Purpose
<code>request</code>	<code>HttpServletRequest</code>	Incoming request data
<code>response</code>	<code>HttpServletResponse</code>	Outgoing response

Object	Type	Purpose
<code>session</code>	<code>HttpSession</code>	Per-user session
<code>application</code>	<code>ServletContext</code>	App-wide shared data
<code>out</code>	<code>JspWriter</code>	Write output to the page
<code>config</code>	<code>ServletConfig</code>	Servlet init parameters
<code>pageContext</code>	<code>PageContext</code>	Access to all other scopes
<code>page</code>	<code>Object</code> (this)	The current JSP instance
<code>exception</code>	<code>Throwable</code>	Only in error pages (<code>isErrorPage="true"</code>)

```
<html><body>
<%
    String name = request.getParameter("name");
    session.setAttribute("user", name);
%>
<h2>Hello, <%= name %>! Session ID: <%= session.getId() %></h2>
</body></html>
```

Exam tip: There are **9 implicit objects** total — list all 9 with their types when asked, since partial answers lose easy marks here.

 **GFG: JSP Implicit Objects**

[↑ TOC](#)

4.2 JSP/Servlet Database Connectivity — Oracle, MS-SQL, MySQL

```
// java.sql package — the standard JDBC API, same regardless of DB vendor
import java.sql.*;

public class DbConnectExamples {
    // MySQL
    static Connection connectMySQL() throws SQLException {
        return DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/mydb", "root", "password");
    }

    // Oracle
    static Connection connectOracle() throws SQLException {
        return DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:XE", "system", "password");
    }

    // MS-SQL Server
    static Connection connectMSSQL() throws SQLException {
```

```

        return DriverManager.getConnection(
            "jdbc:sqlserver://localhost:1433;databaseName=mydb", "sa",
            "password");
    }
}

```

Explain the point being tested here: The JDBC API (`java.sql.*`) is identical for every database vendor — only the **connection URL** and the **driver JAR** on the classpath change (e.g., `mysql-connector-j-x.x.x.jar`, `ojdbc11.jar`, `mssql-jdbc-x.x.jar`, placed in `WEB-INF/lib` for a web app). This vendor-independence is JDBC's core design goal.

Querying, Inserting, Deleting, Modifying Records

```

try (Connection conn = DriverManager.getConnection(url, user, pass)) {

    // SELECT
    try (PreparedStatement ps = conn.prepareStatement("SELECT * FROM
students WHERE cgpa > ?")) {
        ps.setFloat(1, 8.0f);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) System.out.println(rs.getString("name"));
    }

    // INSERT
    try (PreparedStatement ps = conn.prepareStatement(
        "INSERT INTO students (name, cgpa) VALUES (?, ?)")) {
        ps.setString(1, "Anjali");
        ps.setFloat(2, 9.1f);
        ps.executeUpdate();
    }

    // UPDATE
    try (PreparedStatement ps = conn.prepareStatement(
        "UPDATE students SET cgpa = ? WHERE name = ?")) {
        ps.setFloat(1, 9.5f);
        ps.setString(2, "Anjali");
        ps.executeUpdate();
    }

    // DELETE
    try (PreparedStatement ps = conn.prepareStatement("DELETE FROM students
WHERE name = ?")) {
        ps.setString(1, "Anjali");
        ps.executeUpdate();
    }
}

```

Types of Statement

Type	Use Case
<code>Statement</code>	Static SQL, no parameters — vulnerable to SQL injection, avoid for user input
<code>PreparedStatement</code>	Precompiled, parameterized (<code>?</code> placeholders) — faster on repeat, injection-safe
<code>CallableStatement</code>	Calls stored procedures (<code>{call procName(?, ?)}</code>)

Why `PreparedStatement` prevents SQL injection (explain the mechanism, not just the fact): With `Statement`, user input is concatenated directly into the SQL string, so malicious input like `' OR '1'='1` can alter the query's logic. With `PreparedStatement`, the SQL query structure is compiled *first* with placeholders (`?`), and the user-supplied values are then bound to those placeholders as pure *data* — they can never be interpreted as SQL syntax, which is what blocks injection attacks.

 **GFG:** [JDBC PreparedStatement vs Statement](#)

[↑ TOC](#)

4.3 Separating Business Logic and Presentation Logic

```
Bad (logic mixed into JSP):
    <% Connection conn = DriverManager.getConnection(...); ResultSet rs =
...; %>
    <table><% while(rs.next()) { %><tr>...</tr><% } %></table>

Good (JavaBean/DAO handles logic, JSP only PRESENTS):
    Servlet/Controller → calls a StudentDAO (business logic) → stores result
as a request attribute
    JSP                → simply loops over the attribute and renders HTML
(no direct DB code)
```

Why this separation matters (theory point, often asked): Mixing database/business logic directly into JSP makes the page hard to read, hard to maintain, and impossible to reuse the logic elsewhere — it violates the *separation of concerns* principle. Moving logic into JavaBeans/DAO classes (and orchestrating them from a Servlet Controller) means the JSP's only job is rendering — this is the foundational idea behind the **MVC pattern** ([Section 5.1](#)).

Building and Using a JavaBean

Definition: A JavaBean is a plain, reusable Java class that follows specific conventions: a public **no-argument constructor**, **private fields**, **public getter and setter** methods for each property, and (ideally) implementing `Serializable`.

```
// StudentBean.java
package com.rohit.beans;

import java.io.Serializable;
```

```
public class StudentBean implements Serializable {
    private String name;
    private float cgpa;

    public StudentBean() { }    // required no-arg constructor

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public float getCgpa() { return cgpa; }
    public void setCgpa(float cgpa) { this.cgpa = cgpa; }
}
```

```
<jsp:useBean id="student" class="com.rohit.beans.StudentBean"
scope="page"/>
<jsp:setProperty name="student" property="*" />    <!-- auto-binds matching
request params -->

<p>Name: <jsp:getProperty name="student" property="name" /></p>
<p>CGPA: <jsp:getProperty name="student" property="cgpa" /></p>
```

JavaBean naming convention (list this for full marks): For a private field `name`, the getter must be `getName()` (or `isName()` for a `boolean`) and the setter must be `setName(String name)` — this strict naming convention is what allows tools like `<jsp:setProperty property="*" />` to automatically match incoming request parameters to bean properties via reflection.

 [GFG: JavaBeans in Java](#)

[↑ TOC](#)

4.4 Session Handling in JSP

Identical mechanisms to Servlets ([Section 3.5](#)), accessed via the implicit `session` object:

```
<%
    session.setAttribute("username", request.getParameter("username"));
    session.setMaxInactiveInterval(30 * 60);    // 30 minutes, in seconds
%>
Welcome back, <%= session.getAttribute("username") %>!
```

 [GFG: Session Handling in JSP](#)

[↑ TOC](#)

4.5 Types of Errors and Exception Handling (JSP)

```

<!-- errorPage.jsp -->
<%@ page isErrorPage="true" %>
<h2>An error occurred: <%= exception.getMessage() %></h2>

<!-- source.jsp - declares which page handles ITS errors -->
<%@ page errorPage="errorPage.jsp" %>
<%
    int x = 10 / 0;    // throws ArithmeticException - forwarded to
errorPage.jsp
%>

```

Three categories of JSP errors (list these clearly — a standard theory question):

1. **Translation-time errors** — occur when the JSP file itself has bad syntax (mismatched tags, invalid directives), so the container fails to translate it into a servlet at all.
2. **Compilation-time errors** — occur when the embedded Java code (inside scriptlets/expressions) is syntactically invalid, so the auto-generated servlet source fails to compile.
3. **Runtime exceptions** — occur while the servlet is executing (e.g., `NullPointerException`, `SQLException`, `ArithmeticException`); these are handled using `errorPage/isErrorPage` attributes for a specific page, or the `<error-page>` element in `web.xml` for status-code-based handling (like custom 404/500 pages) across the whole application.

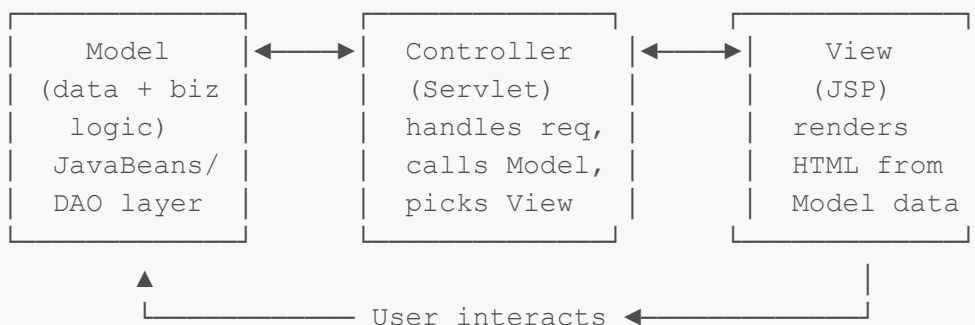
 **GFG:** [Exception Handling in JSP](#)

[↑ TOC](#)

UNIT-V: MVC, RMI & EJB

5.1 MVC Architecture

Definition: MVC (Model-View-Controller) is a software design pattern that separates an application into three interconnected components, so that each can be developed, tested, and modified independently.



Explain each component (write this exactly in exam):

- **Model** — represents the application's data and business logic, implemented as JavaBeans and DAO (Data Access Object) classes. It has no knowledge of how the data is displayed.
- **View** — the presentation layer, implemented as JSP pages; purely responsible for rendering HTML from data it is given ([Section 4.3](#)) and contains no business logic.
- **Controller** — implemented as a Servlet; it receives the incoming HTTP request, invokes the appropriate Model method(s) to perform business logic, and then forwards control to the appropriate View using `RequestDispatcher.forward()`.

```
@WebServlet("/students")
public class StudentController extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        StudentDAO dao = new StudentDAO(); // Model
        List<StudentBean> students = dao.getAllStudents();
        req.setAttribute("students", students);
        req.getRequestDispatcher("studentList.jsp").forward(req, resp); //
View
    }
}
```

Benefits of MVC (list all four for full marks on "advantages of MVC"):

1. **Separation of concerns** — each layer has one clear responsibility, making the codebase easier to understand.
2. **Easier testing** — business logic (Model) can be unit-tested independently of the UI.
3. **Parallel development** — frontend developers can work on Views while backend developers work on the Model/Controller simultaneously.
4. **Reusability** — the same Model can be reused with different Views (e.g., an HTML view and a mobile app view).

 **GFG:** [MVC Architecture in Java](#)

[↑ TOC](#)

5.2 Introduction to Remote Method Invocation (RMI)

Definition: RMI (Remote Method Invocation) is a Java API that allows an object running in one JVM to invoke a method on an object running in a *different* JVM — potentially on a completely different physical machine — making the remote call look and behave like a normal local method call.

```
// 1. Remote interface
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface GreetingService extends Remote {
    String greet(String name) throws RemoteException;
}
```

```
// 2. Server-side implementation
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class GreetingServiceImpl extends UnicastRemoteObject implements
GreetingService {
    protected GreetingServiceImpl() throws RemoteException { super(); }

    @Override
    public String greet(String name) throws RemoteException {
        return "Hello, " + name + "! (from the remote server)";
    }
}

// 3. Server — registers the object with the RMI registry
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Server {
    public static void main(String[] args) throws Exception {
        GreetingServiceImpl service = new GreetingServiceImpl();
        Registry registry = LocateRegistry.createRegistry(1099);
        registry.rebind("GreetingService", service);
        System.out.println("RMI server ready...");
    }
}

// 4. Client — looks up and calls the remote object
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.getRegistry("localhost", 1099);
        GreetingService service = (GreetingService)
registry.lookup("GreetingService");
        System.out.println(service.greet("Rohit"));
    }
}
```

RMI Architecture (draw and explain — a standard diagram question):

```
Client → Stub (local proxy) → Network → Skeleton/Registry → Remote Object
(Server)
```

Explain the roles:

- **Stub** — a client-side proxy object that has the same interface as the remote object. When the client calls a method on the stub, it packages (*marshals*) the call details and sends them over the network.

- **Skeleton / Registry** — on the server side, receives the network call, unpacks (*unmarshals*) it, and invokes the actual method on the real remote object; the **RMI Registry** is a simple lookup service (like a phone book) where server objects are *bound* under a name and clients *look them up* by that name.

Steps to build an RMI application (standard 4-step answer to memorize):

1. Define a **remote interface** that extends `java.rmi.Remote` and declares methods that each throw `RemoteException`.
2. Implement that interface in a **server class**, typically extending `UnicastRemoteObject`.
3. Register the implementation object with the **RMI Registry** on the server (`Registry.rebind()`).
4. On the client, **look up** the remote object by name via the registry and invoke its methods as if it were local.

 **GFG: RMI in Java**

[↑ TOC](#)

5.3 Introduction to Enterprise JavaBeans (EJB)

Definition: EJB (Enterprise JavaBean) is a server-side, managed component model (part of Jakarta EE) used to build distributed, transactional, and secure business logic — the EJB **container** automatically handles cross-cutting concerns like concurrency, transactions, security, and object lifecycle, so the developer can focus purely on writing business logic.

Types of EJB

- ```
1. Session Bean
 ├── Stateless — no client-specific state retained between calls (pooled
 & reused, most scalable)
 ├── Stateful — retains conversational state for ONE client across
 multiple calls
 └── Singleton — exactly ONE instance shared application-wide

2. Message-Driven Bean (MDB) — invoked asynchronously in response to JMS
 messages, not direct calls

(Entity Beans existed in EJB 2.x for O/R mapping — deprecated in favor of
JPA in modern Jakarta EE)
```

### Explain each Session Bean type with a real-world analogy (helps in long-answer questions):

- **Stateless Session Bean** — like an ATM machine that doesn't remember you between transactions; each call is independent, so the container can freely reuse instances from a pool across different clients — this makes it the most scalable EJB type. Example: a `CalculatorService` that just adds two numbers.
- **Stateful Session Bean** — like a personal shopping cart that remembers what you've added across multiple visits to the same session; the container dedicates one bean instance per client for as long as that conversation lasts.

- **Singleton Session Bean** — exactly one shared instance exists for the *entire application*, used for data or logic that must be common to every client (e.g., an application-wide cache or counter).
- **Message-Driven Bean (MDB)** — unlike Session Beans, it is never called directly by a client; instead it's triggered *asynchronously* when a message arrives on a JMS (Java Message Service) queue/topic — useful for decoupled, event-driven processing.

## Creating and Working with a Session Bean

```
// Remote/Local business interface
import jakarta.ejb.Local;

@Local
public interface CalculatorService {
 int add(int a, int b);
}

// Stateless Session Bean implementation
import jakarta.ejb.Stateless;

@Stateless
public class CalculatorServiceBean implements CalculatorService {
 @Override
 public int add(int a, int b) {
 return a + b;
 }
}

// Stateful Session Bean example — retains state ACROSS calls for one
// client
import jakarta.ejb.Stateful;

@Stateful
public class ShoppingCartBean {
 private java.util.List<String> items = new java.util.ArrayList<>();

 public void addItem(String item) { items.add(item); }
 public java.util.List<String> getItems() { return items; } //
 remembers items across multiple calls
}

// Injecting and using an EJB from a Servlet (in a Jakarta EE container)
import jakarta.ejb.EJB;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/calc")
public class CalcServlet extends HttpServlet {
 @EJB
 private CalculatorService calculator; // container injects the bean
 automatically
}
```

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
throws IOException {
 int result = calculator.add(5, 10);
 resp.getWriter().println("Result: " + result);
}
}
```

### Container-provided services for EJBs (list all for full marks on "services provided by EJB container"):

1. **Declarative transaction management** (`@TransactionAttribute`) — transactions are configured, not hand-coded.
2. **Declarative security** (`@RolesAllowed`) — access control rules are attached as annotations rather than written manually.
3. **Instance pooling & lifecycle management** — the container creates, pools, and destroys bean instances automatically.
4. **Remote/local invocation transparency** — calling a bean looks the same whether it's local or on a remote server.
5. **Dependency injection** (`@EJB`, `@Resource`) — the container automatically wires up beans and resources, removing manual lookup code.

### RMI vs EJB (a good comparison to add if a question compares distributed technologies):

| RMI                                                                  | EJB                                                                            |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Low-level remote invocation mechanism                                | High-level, managed component model built on top of remote-invocation concepts |
| No built-in transaction/security management — must be coded manually | Container automatically provides transactions, security, pooling               |
| Developer manages the object's lifecycle                             | Container manages the bean's lifecycle                                         |

 **GFG:** [Enterprise Java Beans \(EJB\)](#)

[↑ TOC](#)



## Recommended Resources (Books + Official Docs)

| Resource                                                                                              | Covers                  |
|-------------------------------------------------------------------------------------------------------|-------------------------|
| <i>Java: The Complete Reference</i> — Herbert Schildt                                                 | Unit I & II             |
| <i>Head First Servlets &amp; JSP</i>                                                                  | Unit III & IV           |
| <i>Head First EJB</i> / Oracle Jakarta EE Tutorial                                                    | Unit V                  |
| <a href="https://docs.oracle.com/javase">docs.oracle.com/javase</a> [Verified — official Oracle docs] | Core Java API reference |

| Resource                                                                         | Covers                                           |
|----------------------------------------------------------------------------------|--------------------------------------------------|
| <a href="#">Jakarta EE Specifications</a> [Verified — official spec]             | Servlet, JSP, EJB specs                          |
| <a href="#">Apache Tomcat Docs</a> [Verified — official Tomcat docs]             | Server setup & deployment                        |
| <a href="#">GeeksforGeeks Java Tutorial (hub page)</a> [Verified — GFG Java hub] | Everything, unit-by-unit with practice questions |

[↑ TOC](#)

## Quick Exam-Day Revision Checklist

Before the exam, make sure you can write these from memory (they cover ~70% of typical M.Tech Java papers):

- ☐ [Features of Java](#) (top 6 with 2 lines each)
- ☐ [JDK vs JRE vs JVM](#)
- ☐ [Primitive data types table](#) (size, default, range)
- ☐ [JVM memory areas diagram](#) (Heap, Stack, Method Area, PC Register, Native Stack)
- ☐ [Widening vs Narrowing conversion](#)
- ☐ [String vs StringBuffer vs StringBuilder table](#)
- ☐ [Why String is immutable](#) (4 reasons) — [see 1.12](#)
- ☐ [Method overloading resolution order](#)
- ☐ [Access modifier table](#) (private/default/protected/public)
- ☐ [Checked vs Unchecked exceptions table](#)
- ☐ [Thread creation: extends Thread vs implements Runnable](#)
- ☐ [Thread life cycle diagram + states table](#)
- ☐ [JDBC driver types \(1–4\) and steps to connect](#)
- ☐ [Web Server vs Application Server](#)
- ☐ [Servlet life cycle: init\(\) → service\(\) → destroy\(\)](#)
- ☐ [Session tracking: all 4 methods](#)
- ☐ [JSP implicit objects \(all 9\)](#)
- ☐ [Statement vs PreparedStatement vs CallableStatement](#)
- ☐ [MVC architecture diagram + role of each layer](#)
- ☐ [RMI architecture \(Stub/Skeleton/Registry\) + 4 steps to build an RMI app](#)
- ☐ [Types of EJB \(Stateless/Stateful/Singleton/MDB\) with analogies](#)

[↑ TOC](#)

*Java M.Tech Exam-Ready Notes | Units I–V | Core Java → Servlets/JSP → RMI/EJB | GFG reference links added per topic for extra practice*